



香港中文大學 (深圳)
The Chinese University of Hong Kong

CSC 5003 Algorithm Art and Programming Practice Lecture 7: Adhoc & Mid-term Review

Jingbang Chen
School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen



What do we learn before mid-term?

- ▶ STL
 - Use sort/map/set/lower(upper)-bound properly.
 - Treat them as a tool when needed.
- ▶ Trees and Graphs: DFS & BFS
 - Decompose graphs into trees
 - Decompose trees into paths
 - Decompose cycles into chains



What do we learn before mid-term?

- ▶ **Topsort and Shortest Path**
 - Focus on model the problem into graphs:
 - Preference and order → directed graphs
 - Operation cost → edge weights
 - Ensure the target matches the real goal.
- ▶ **String Processing**
 - KMP & Manacher
 - Hashing is powerful!
- ▶ **Search**
 - Use the right framework (BFS/DFS)
 - DFS for searching up the whole space
 - BFS for shortest paths
 - BFS: Compress necessary information into a state
 - DFS: Choose what to enumerate wisely



3/15 1:30 – 4:30

- ▶ At least 2 problems of Midterm are selected from the candidate sets.
- ▶ At most 1 problem from today.
- ▶ 1 problem not from class
- ▶ 4 problems in total.

- ▶ You are only allowed to use the desktop, **NOT YOUR OWN COMPUTER!**

- ▶ Bring you ID for identity verification.

- ▶ Ensure to check your score when leaving the room, we will confirm it with your signature.



Picture Rotation

- ▶ Given an $n \times n$ matrix, rotate it clockwise for 90 degree.
- ▶ Bonus Requirement: You should not use any extra space.

示例 1:

1	2	3
4	5	6
7	8	9



7	4	1
8	5	2
9	6	3

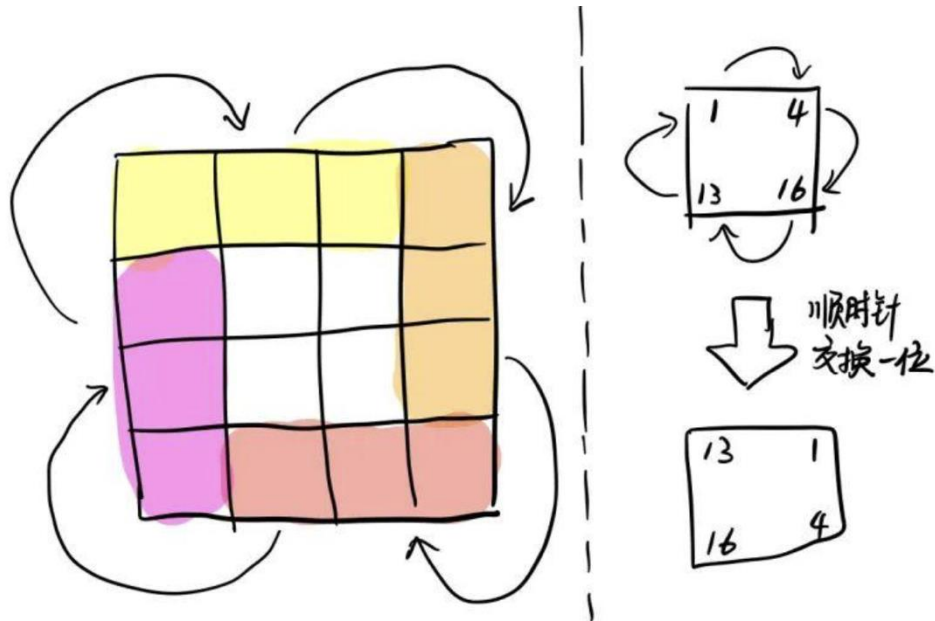


Solution

► Solution 1:

Use extra space, mapping accordingly to a new matrix, and then copy it back.

► Solution 2:





Solution

► Solution 3:

1	2	3
4	5	6
7	8	9



1	4	7
2	5	8
3	6	9



7	4	1
8	5	2
9	6	3



A Number that Exist Once

Version 1: Every other number exist exactly twice.

Version 2: Every other number exist exactly three times.

Version 3: Two numbers that occur once while other exist exactly twice.

Interview: Don't use hashmap!!!!!!!!!!!!!!!!!!!!!!



Solution: Find it/them bitwisely!

- ▶ Version 1: Modulo 2
 - Wait....this is called bitwise exclusive (xor)

- ▶ Version 2: Modulo 3
 - Now we can do for arbitrary k ?

- ▶ Version 3:
 - Since a is not b , then find a bit that they are different.
 - Do it with version 1
 - Split to two groups
 - Find them with version 1



Majority Element

- ▶ Given n elements, find an element that occurs more than $\lfloor n/2 \rfloor$ times.
- ▶ Given n elements, find an element that occurs more than $\lfloor n/3 \rfloor$ times.
- ▶ You need to do it in $O(n)$ time, $O(1)$ space!



Moore Voting

- ▶ Imagine as an election:
 - For the current candidate x , set a counting cnt
 - For any incoming ticket:
 - If cnt is 0, change x to the current voting target.
 - if the ticket goes to x , cnt increase by 1.
 - Otherwise decrease by 1.
- ▶ The final x is our majority element.
- ▶ Proof: Since we know cnt tickets go to x in last. If x is not the majority element, then the remaining $n - \text{cnt}$ tickets can select the right answer. However, we know each of these tickets are being eliminated by two different candidates. Then a candidate within can get at most $(n - \text{cnt})/2$ tickets, which is less than $n/2$. Thus x must be the majority element.



Moore Voting

- ▶ Bonus: What about the $\lfloor n/3 \rfloor$ case? What about $\lfloor n/m \rfloor$ case?
- ▶ Since there are at most two elements with more than $n/3$ occurrences. We maintain $x1$ and $x2$, $cnt1$ and $cnt2$:
 - If one of $x1$ and $x2$ is 0, update one of them.
 - If the ticket goes to $x1$ or $x2$, increase $cnt1$ and $cnt2$ accordingly.
 - Otherwise decrease both $cnt1$ and $cnt2$ by 1.



Maximum Equal Frequency

- ▶ Find the longest prefix such that:
 - By removing exactly one element, all remaining elements have the same frequency.
- ▶ $N \leq 10^5$
- ▶ Hint: Think about which element we should remove once the prefix is fixed.



Solution

- ▶ Case 1: All elements occur exactly once.
- ▶ Case 2: All but one occur k times while one occurs $k+1$ time.
- ▶ Case 3: All but one occur k times while one occurs 1 time.
- ▶ Keep track of the maximum occurrence and corresponding elements.
- ▶ With a hashmap, the complexity is $O(n)$.



Next Permutation

- ▶ Find the next permutation or determine there is not.
- ▶ Don't cheat with existing functions!



Solution

- ▶ Imagine it is a long integer.
- ▶ We want to find the key element to swap in something larger.
 - From back to front: find the first $a[i] < a[i+1]$.
 - Find the minimum increasement: find the smallest $a[j] > a[i]$
 - Swap $a[i]$ and $a[j]$.
- ▶ We want ensure $i+1..n$ is smallest possible:
 - Sort (reverse) them.



Summary

- ▶ Adhoc problems are hard because you don't know where to start from.
- ▶ Adhoc problems are hard because it often requires case analysis.
- ▶ Key: See more and summarize more!